AutoSage App Using Gemini Flash

Team ID : LTVIP2026TMIDS91131

Team Size : 4

Team Leader : Eedara Tejaswini

Team member : Sai Kumar Ramoju

Team member : Prasanna Kumar Raparla

Team member : Sandeep Peruri

1. INTRODUCTION

1.1 Project Overview

AutoSage is a multimodal AI-powered vehicle intelligence application developed using Google Gemini 2.5 Flash and Streamlit. The primary objective of the system is to analyze vehicle images and generate structured automotive insights tailored to the Indian automobile market. The system leverages the multimodal capabilities of Gemini Flash to process image inputs and generate contextual and structured responses.

Unlike traditional automotive platforms that rely on manual searches or static databases, AutoSage uses generative AI to interpret vehicle images and provide detailed information such as brand, model, fuel type, engine capacity, mileage, market price range, maintenance level, safety features, and estimated resale value.

The application is divided into two main modules:

- **Primary Module – Image-Based Vehicle Analysis**
  This module allows users to upload vehicle images and receive structured vehicle insights generated using multimodal AI reasoning.
- **Enhancement Module – AutoSage Conversational Assistant**
  This module allows users to interact with the system through

text queries for vehicle recommendations, comparisons, maintenance advice, and eco-friendly vehicle options. The system integrates AI reasoning with structured prompt engineering to ensure output consistency and practical usefulness.

1.2 Purpose

The purpose of AutoSage is to design and implement an intelligent AIbased automotive advisory system capable of:

- Identifying vehicles from uploaded images.
- Generating structured and decision-focused vehicle insights.
- Providing price estimation and resale value guidance.
- Supporting buyers in making informed decisions.
- Offering an additional conversational assistant for enhanced usability.

The project demonstrates the real-world application of multimodal generative AI in automotive intelligence systems.

2. IDEATION PHASE

2.1 Problem Statement

The automobile market contains thousands of vehicle models and variants. Users often face difficulties in:

- Identifying vehicle models from images.
- Comparing similar vehicle variants.
- Understanding price differences.
- Estimating long-term resale value.
- Evaluating maintenance costs.
- Finding eco-friendly alternatives.

Existing platforms provide scattered information across multiple websites, leading to confusion and time consumption.

There is a need for a unified intelligent system that:

- Accepts vehicle images as input.
- Generates structured automotive insights.
- Provides contextual Indian market information.
- Supports interactive advisory capabilities.

AutoSage addresses this gap by integrating multimodal AI for structured vehicle intelligence.

2.2 Empathy Map Canvas

Target Users:

- First-time vehicle buyers.
- Used car buyers.
- Automobile enthusiasts.
- Eco-conscious customers.
- Individuals seeking maintenance guidance.

What Users Think:

- "Is this vehicle worth the price?"
- "How much resale value will it have?"
- "Which vehicle suits my budget?"
- "Is this vehicle fuel-efficient?" **What Users Feel:**
- Confused due to too many choices.
- Overwhelmed by technical specifications.
- Concerned about long-term maintenance.

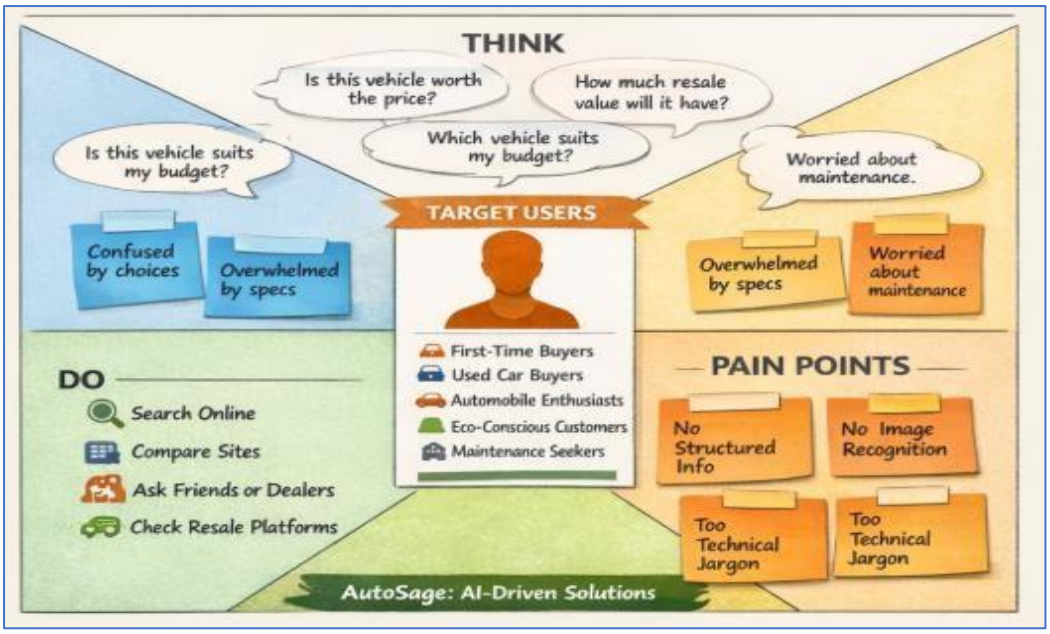
What Users Do:

- Search online.
- Compare multiple websites.
- Ask friends or dealers.
- Check resale platforms.

Pain Points:

- Lack of structured insights.
- No image-based identification.
- Complex technical language.

AutoSage simplifies this process through AI-driven structured output.



2.3 Brainstorming

During the ideation phase, multiple technical approaches were evaluated:

Approach 1: Static Database Lookup

Limitation: Requires continuous manual updates.

Approach 2: Traditional Image Classification (CNN)

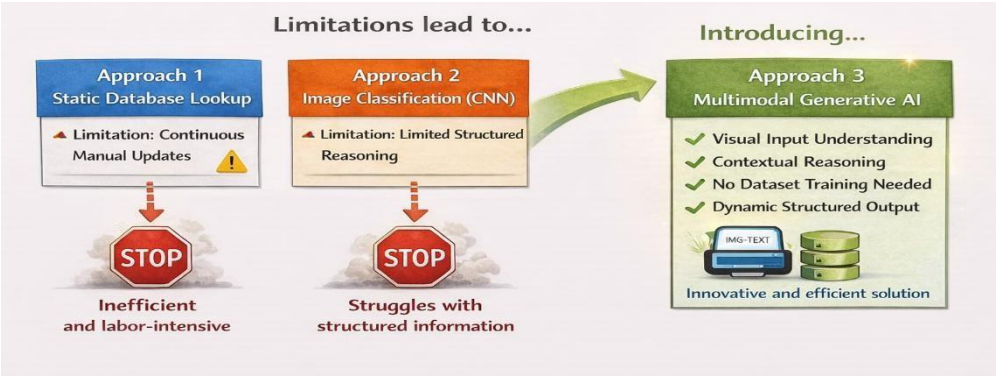
Limitation: Can classify brand/model but cannot generate structured reasoning.

Approach 3: Multimodal Generative AI Advantage:

- Understands visual input.
- Generates contextual explanations.
- No need for dataset training.
- Supports dynamic reasoning.

Final Decision:

Implement a multimodal AI system using Gemini 2.5 Flash with structured prompt engineering.



3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

1. User opens AutoSage application.
2. User uploads a vehicle image.
3. System processes image using Gemini API.
4. Structured automotive insights are displayed.
5. User optionally interacts with chat assistant.
6. User makes an informed decision.

3.2 Solution Requirement

Functional Requirements:

- Image upload functionality.
- Image processing and byte extraction.
- Structured prompt generation.
- API integration with Gemini.
- Response formatting and display.
- Chat assistant interaction.
- Session state management.

Non-Functional Requirements:

- Response time below 5 seconds.
- Secure API key handling.
- Structured output formatting.
- User-friendly interface.

3.3 Data Flow Diagram

The AutoSage system follows a dual-layer data processing architecture consisting of a primary image-based analysis flow and a secondary text-based advisory flow. Both flows are integrated with the Gemini 2.5 Flash AI Engine.

Primary Flow – Image-Based Vehicle Analysis

1. The user uploads a vehicle image through the Streamlit interface.
2. The image is processed in the **Vehicle Image Analysis Layer**, where image bytes and format are prepared for AI processing.
3. The processed image, along with a structured prompt, is sent to the **Gemini 2.5 Flash AI Engine**.
4. The AI engine analyzes the image and generates structured automotive details.

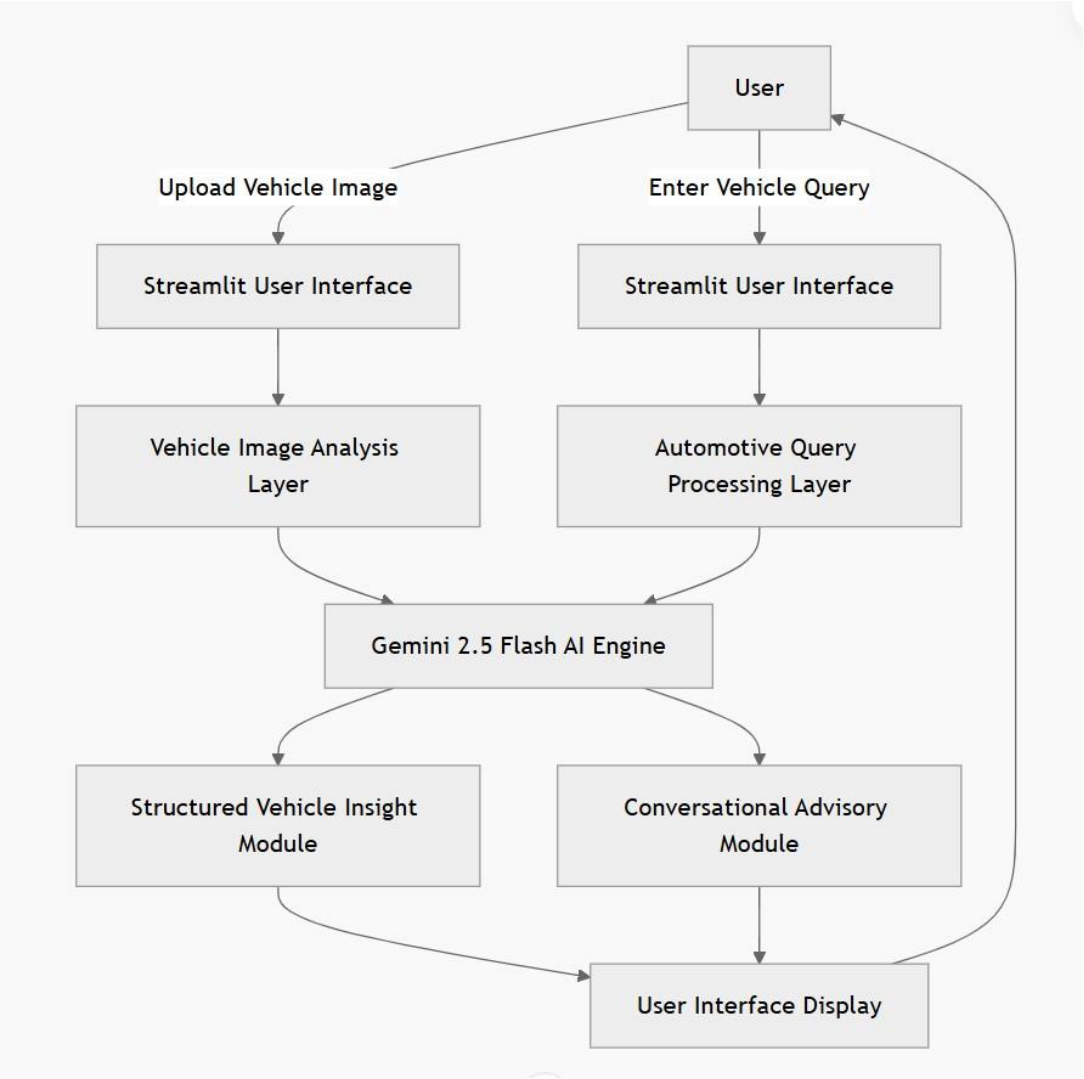
5. The response is organized in the **Structured Vehicle Insight Module** and displayed to the user.

This flow represents the core functionality of AutoSage.

Secondary Flow – Text-Based Advisory (Enhancement)

1. The user enters a vehicle-related query.
2. The query is processed in the **Automotive Query Processing Layer**, where it is converted into a structured prompt.
3. The prompt is sent to the **Gemini 2.5 Flash AI Engine**.
4. The AI generates an advisory response.
5. The output is managed by the **Conversational Advisory Module** and displayed in chat format.

This module enhances the system by providing recommendations, comparisons, and maintenance guidance.



3.4 Technology Stack

- **Programming Language: Python**
  Python is used as the core programming language for developing the AutoSage application. It provides strong support for AI integration, API communication, and rapid web application development. Its simplicity and extensive library ecosystem make it ideal for integrating machine learning models and handling backend logic efficiently.

- **Frontend Framework: Streamlit**
  Streamlit is used to build the interactive web interface of the application. It enables rapid UI development with minimal code and supports dynamic components such as image upload, chat input, buttons, and structured output display. Streamlit also allows seamless deployment on cloud platforms with minimal configuration.
- **AI Model: Gemini 2.5 Flash**
  Google Gemini 2.5 Flash is the multimodal AI model used in the project. It supports both image and text inputs, making it suitable for vehicle image analysis and conversational advisory features. The model provides fast response times and contextual reasoning, which is essential for generating structured automotive insights.
- **Environment Handling: dotenv**
  The python-dotenv library is used to securely manage environment variables, particularly the Google API key. This prevents sensitive credentials from being hardcoded into the source code and ensures safer deployment practices.
- **Image Processing: PIL (Pillow)**
  Pillow (PIL) is used for handling uploaded images. It processes image files, ensures correct formatting, and converts image data into byte format suitable for API input. This step is essential for enabling Gemini's multimodal capabilities.
- **Deployment: Localhost / Cloud Deployment Ready**
  The application is designed to run locally during development using Streamlit. It is also cloud-ready and can be deployed on platforms such as Streamlit Cloud or other hosting environments, ensuring scalability and accessibility.

4. PROJECT DESIGN

4.1 Problem Solution Fit

AutoSage directly solves the problem of:

- Vehicle identification from images.
- Structured automotive data generation.
- Intelligent decision-making support.

The system uses structured prompts to control AI output format, ensuring consistent and professional responses.

4.2 Proposed System

The system is divided into two layers:

Primary Layer – Image Analysis

- Upload vehicle image.
- Send image + structured prompt to Gemini.
- Generate structured vehicle report.

Secondary Layer – Chat Assistant

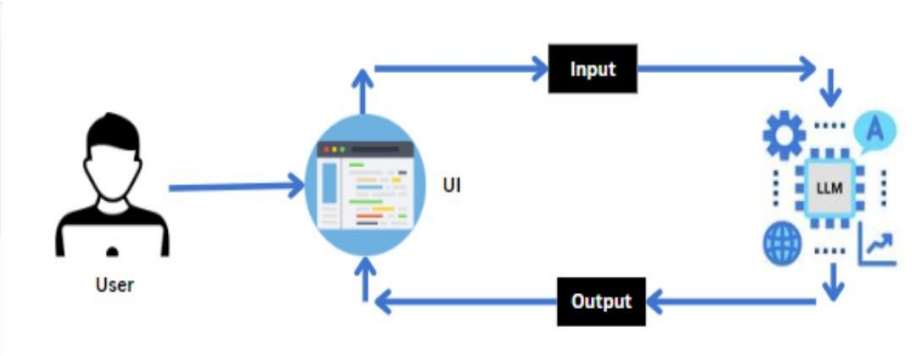
- Accept user queries.
- Provide recommendations.
- Compare vehicles.
- Offer maintenance tips.
- Suggest EV options.

4.3 Solution Architecture

The AutoSage system follows a simplified yet effective client–AI architecture model. The architecture focuses on seamless interaction between the user interface and the Gemini 2.5 Flash multimodal AI model.

The overall architecture consists of four major components:

1. **User Layer** 2. **User Interface (UI) Layer** 3. **Processing Layer (LLM – Gemini Flash)** 4. **Output Layer**



Architecture Flow Explanation Step 1: User Interaction

The process begins when the **user interacts with the system** through the web interface. The user can:

- Upload a vehicle image (Primary Module)
- Enter a text query (Enhancement Module)

This input is captured through the Streamlit-based UI.

Step 2: Input Processing

The UI forwards the input to the processing component.

- In image mode, the uploaded image is converted into byte format.
- In text mode, the query is structured using predefined prompt engineering logic.

The input is then prepared and transmitted to the Gemini 2.5 Flash model.

Step 3: LLM Processing (Gemini 2.5 Flash)

The Gemini 2.5 Flash model acts as the core intelligence engine of the system.

It performs:

- Multimodal image understanding
- Context-based reasoning
- Structured output generation
- Automotive domain inference

The model processes the input and generates a structured response based on the prompt instructions.

Step 4: Output Generation

The generated response is returned from the LLM to the UI layer. The UI:

- Formats the response
- Displays structured vehicle information
- Renders advisory responses clearly
- Maintains session-based interaction (for chat mode) The final output is then shown to the user.

5. PROJECT PLANNING AND SCHEDULING

Duration	Report Section	Activities Performed	Deliverables
Day 1 – Day 2	**Introduction & Ideation Phase**	Defined project overview and purpose. Identified key automotive challenges faced by users. Prepared problem statement. Developed empathy map canvas. Conducted brainstorming to finalize image-based AI as primary module and chat assistant as enhancement.	Clear project concept and finalized scope.
Day 3 – Day 4	**Requirement Analysis**	Designed customer journey map. Identified functional and non-functional requirements. Created Data Flow Diagram (DFD). Finalized technology stack including Python, Streamlit, Gemini 2.5 Flash, dotenv, and PIL.	Finalized system requirements and workflow structure.
Day 5 – Day 6	**Project Design**	Established problem-solution fit. Designed primary image-based vehicle analysis module. Designed enhancement chat assistant module. Developed high-level solution architecture.	Complete system design blueprint.
Day 7 – Day 8	**API Setup & Model Integration**	Generated Gemini API key. Configured environment variables using dotenv. Integrated Gemini 2.5 Flash model. Performed initial response testing.	Working AI model integration.
Duration	**Report Section**	**Activities Performed**	**Deliverables**

Day 9 – Day 11	**Core Development (Image Module)**	Developed structured image prompt. Implemented image upload interface using Streamlit. Added image-to-bytes processing. Connected image input to Gemini model. Displayed structured output.	Functional Image-Based AI module.
Day 12 – Day 14	**Chat Assistant Integration**	Designed structured prompt for text-based assistant. Implemented session state handling. Created Robo Assistant activation button. Integrated chat interface with mode switching.	Fully integrated multimodal application.
Day 15 – Day 16	**Functional & Performance Testing**	Tested multiple vehicle image scenarios. Tested recommendation, comparison, and maintenance queries. Optimized output formatting. Resolved UI alignment issues. Managed API quota handling.	Stable and optimized application.
Day 17	**Results & Documentation**	Captured output screenshots. Finalized architecture diagrams. Structured project documentation. Reviewed and refined final code.	Submission-ready project.

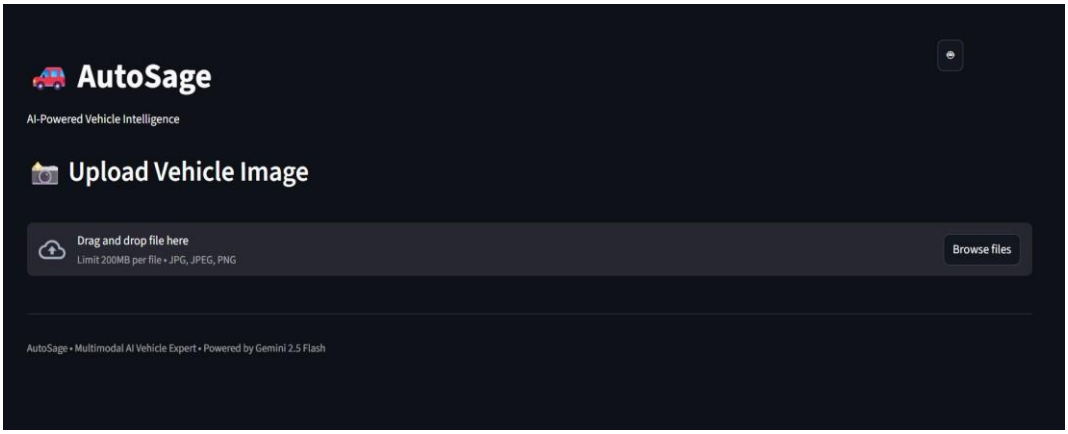
6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Functional Testing

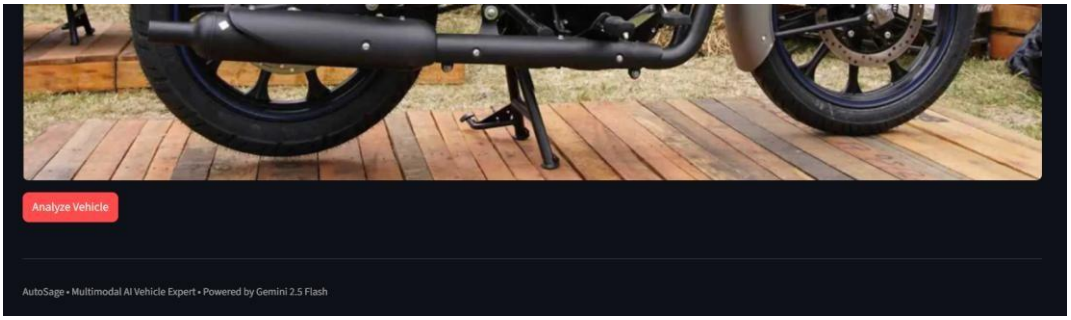
6.1.1 Image Upload Validation

Test Case 1: Analyze Button Visibility Without Image Upload

- Input: No image uploaded
- Expected Result: “Analyze Vehicle” button should not be visible until image is uploaded
- Actual Result: Analyze button appears only after image upload; it is not displayed initially
- Status: Passed



Before Uploading Picture there is no "Analyze Vehicle" button.

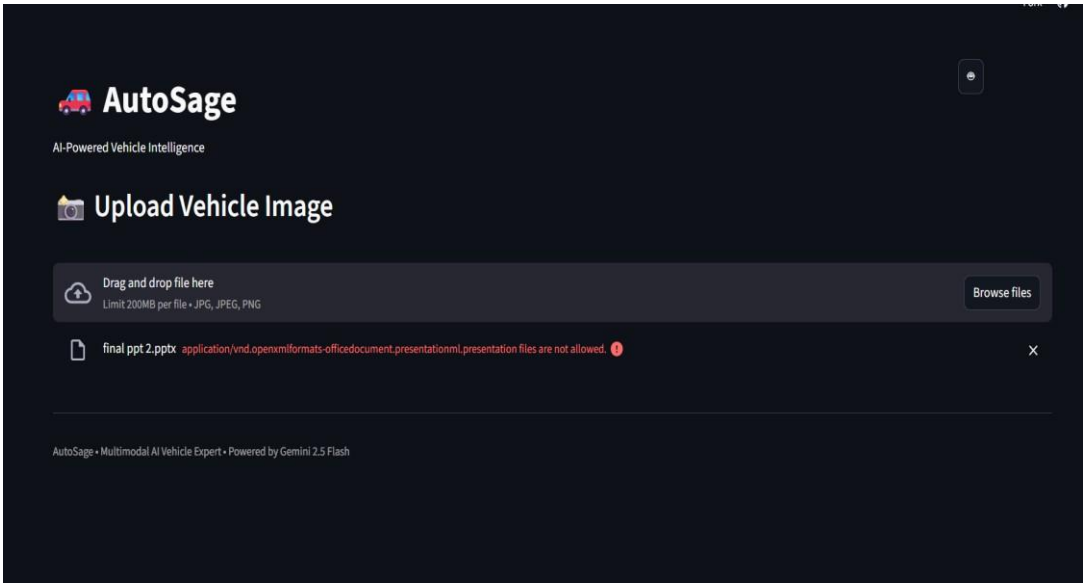


After Uploading Image "Analyze Vehicle" button is enabled.

6.1.2 Image Format Validation

Test Case 2: Invalid File Format Upload

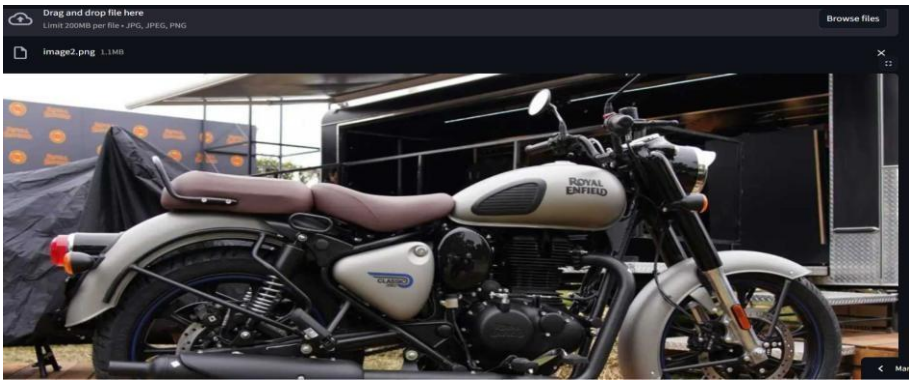
- **Input:** Upload a non-image file (e.g., .pptx file)
- **Expected Result:** System should reject the file and display an error message indicating that the file type is not allowed
- **Actual Result:** Application displays error message:
- **Status:** Passed



6.1.3 Image Upload Functionality

Test Case 3: Unsupported File Format Upload

- Input: JPG/PNG vehicle image
- Expected Result: Image displays correctly
- Actual Result: Image rendered successfully
- Status: Passed



6.1.4 Motorcycle Image Testing

Test Case 4: Motorcycle Image Upload •

Input: Royal Enfield image

- Expected Result:
  - Correct vehicle type
  - Engine capacity shown
  - Maintenance level mentioned
- Actual Result: Accurate structured output generated
- Status: Passed

Vehicle Analysis

Brand: Royal Enfield

Model: Classic 350

Launch Year: 2021 (New J-Series engine model)

Vehicle Type: Motorcycle

Fuel Type: Petrol

Key Features:

- Engine Capacity: 349cc
- Transmission Type: 5-speed manual
- Top 3 Special Features: New J-Series engine, Retro design styling, Tripper Navigation (select variants)

Mileage: 35-40 km/l (Estimated)

Performance:

- Power Output: 20.2 bhp
- Torque: 27 Nm

Average Price in INR: ₹1.93 Lakh - ₹2.25 Lakh (Ex-showroom)

Maintenance Level: Moderate

Safety Features:

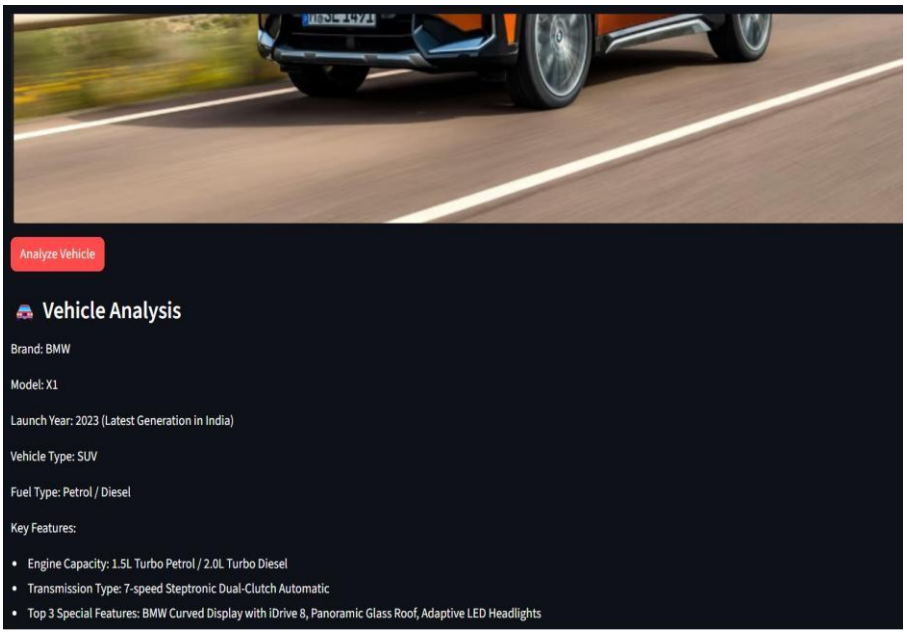
- Dual-channel ABS

6.1.5 Image Analysis Function Test

Case 5: SUV Image Analysis ·

Input: Image of SUV

- Expected Result: ○ Brand detected ○ Model estimated ○ Mileage displayed ○ Price range shown
- Actual Result: Structured vehicle report generated
- Status: Passed

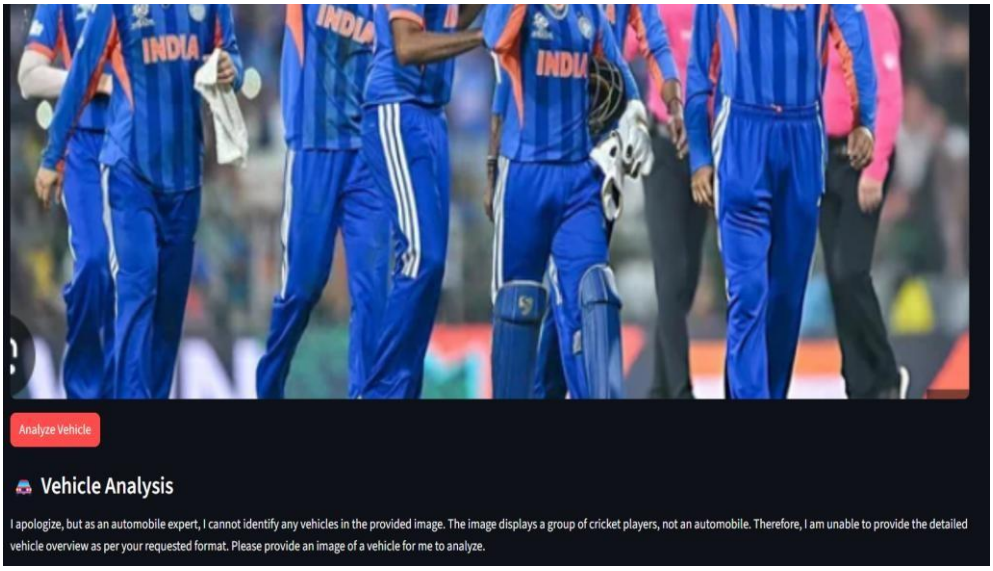


6.1.6 Non-Vehicle Image Handling Test

Case 6: Upload Non-Vehicle Image

- **Input:** Upload an image that does not contain a vehicle (e.g., image of cricket players)

- **Expected Result:** System should identify that no vehicle is present and return an appropriate message requesting a valid vehicle image
- **Actual Result:** System responded: "I cannot identify any vehicles in the provided image… Please provide an image of a vehicle."
- **Status:** Passed



6.1.7 Chat Assistant – Recommendation Query Test

Case 7: Budget-Based Recommendation

- Input: "Best bikes under 2 lakhs"
- Expected Result: ○ 2–4 recommended models ○ Price range shown ○ Mileage mentioned ○ Short verdict included
- Actual Result: Structured recommendation provided
- Status: Passed

You: Best bikes under 2 lakhs

AutoSage: Here are some of the best bike options under 2 lakhs INR, catering to different preferences:

Recommended Models:

1. **TVS Apache RTR 160 4V** - Price Range: INR 1.50 - 1.65 Lakhs (On-road)
   - **Mileage:** 40-45 kmpl
   - **Key Features:**
     - Race-tuned slipper clutch & GTT (Glide Through Traffic)
     - SmartXonnect (Bluetooth) with turn-by-turn navigation
     - Segment-leading power & performance
   - **Best For:** Enthusiastic riders, daily commutes, spirited city riding, and those seeking advanced features.
2. **Bajaj Pulsar N160** - Price Range: INR 1.45 - 1.55 Lakhs (On-road)
   - **Mileage:** 40-45 kmpl
   - **Key Features:**
     - Dual-channel ABS (Segment-first) for enhanced safety
     - Projector LED headlamp
     - Muscular streetfighter design
   - **Best For:** Urban commuting, occasional highway rides, riders prioritizing safety features, and a modern, aggressive look.
3. **Royal Enfield Hunter 350** - Price Range: INR 1.75 - 2.00 Lakhs (On-road)
   - **Mileage:** 35-40 kmpl
   - **Key Features:**
     - Smooth J-series engine

Short Verdict:

- For **performance and features**, the **TVS Apache RTR 160 4V** is a top choice.
- If **safety (Dual-channel ABS) and muscular styling** are priorities, the **Bajaj Pulsar N160** stands out.
- For a blend of **classic appeal, comfort, and an accessible cruiser feel**, the **Royal Enfield Hunter 350** is unmatched.
- For **refined urban commuting, reliability, and good mileage** with modern features, the **Yamaha FZ-S V4** is excellent.

Ultimately, your best choice depends on your riding style, feature preference, and whether you prioritize performance, safety, or classic appeal.

6.1.8 Comparison Query

Test Case 8: Vehicle Comparison •

Input: "Compare Nexon and Brezza"

- Expected Result:
  - Price difference
  - Mileage difference
  - Pros & Cons
- Actual Result: Structured comparison generated
- Status: Passed

You: Compare Nexon and Brezza

AutoSage: Comparison Overview:

Both Tata Nexon and Maruti Suzuki Brezza are popular compact SUVs, offering distinct strengths for the Indian market.

- **Price Difference:**
  - **Tata Nexon:** INR 8.15 - 15.80 Lakh (Petrol, Diesel, EV options available).
  - **Maruti Suzuki Brezza:** INR 8.34 - 14.14 Lakh (Petrol, CNG options available).
  - Nexon offers more powertrain choices (diesel, EV) which broadens its price spectrum.
- **Mileage Difference:**
  - **Tata Nexon (Petrol):** Approx. 17.0-17.4 kmpl. Diesel offers higher.
  - **Maruti Suzuki Brezza (Petrol):** Approx. 17.38-19.89 kmpl (MT/AT). CNG offers significantly higher mileage (approx. 25.51 km/kg).
  - Brezza generally offers better petrol efficiency, and its CNG variant is a clear winner for running costs.
- **Feature Highlights:**
  - **Tata Nexon:**
    - **Top-tier Safety:** 5-star GNCAP safety rating (petrol/diesel).
    - **Feature-rich Cabin:** Ventilated seats, digital instrument cluster, multiple drive modes, bigger touchscreen.
    - **Ride Comfort:** Excellent suspension tuning for varied Indian road conditions.
  - **Maruti Suzuki Brezza:**
    - **Reliable Powertrain:** Smooth and refined K15C engine with good AT.
    - **Fuel Efficiency:** Class-leading mileage, especially with CNG variant.
    - **Maruti Network:** Extensive service network and strong resale value.

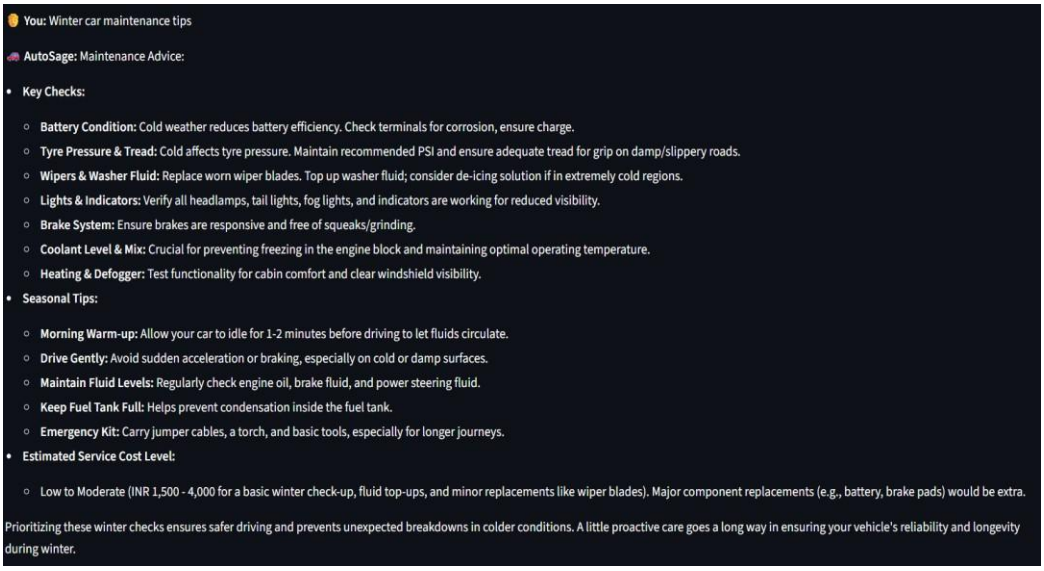
Pros & Cons:

6.1.9 Maintenance Advisory

Test Case 9: Seasonal Maintenance Query •

Input: "Winter car maintenance tips"

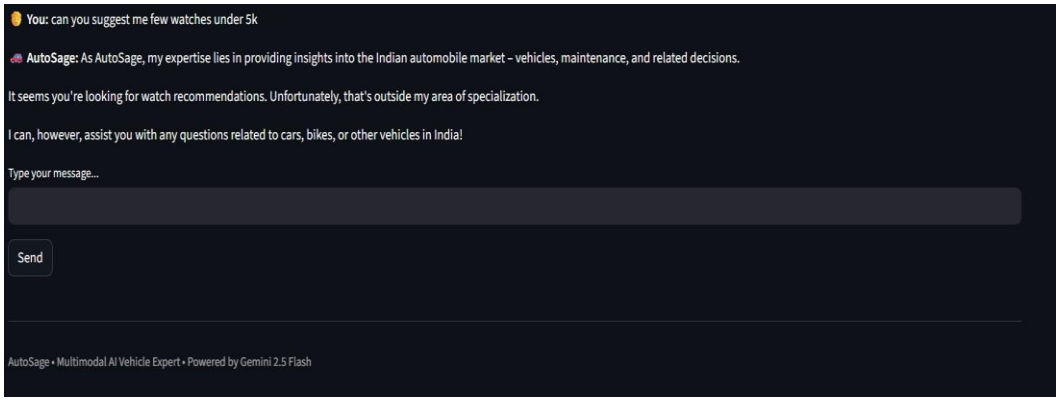
- Expected Result: ○ Tire check ○ Battery check ○ Fluid levels
- Actual Result: Practical maintenance advice provided
- Status: Passed



6.1.10 Domain Restriction Validation

Test Case 10: Non-Automotive Query Handling

- **Input:** "Can you suggest me few watches under 5k?"
- **Expected Result:** System should politely decline and restrict response to automotive-related topics
- **Actual Result:** Assistant responded that watch recommendations are outside its expertise and redirected to vehicle-related queries • **Status:** Passed



6.2 Performance Testing

6.2.1 Response Time Analysis

- Average Image Analysis Response Time: **8–12 seconds**

- Average Chat Assistant Response Time: **7–10 seconds** The response time includes:
- Image byte processing
- Prompt construction
- API communication latency
- AI model inference time

Although the system relies on a cloud-based generative AI model, the response time remained stable and consistent across multiple test cases.

6.2.2 API Stability Testing

- Multiple consecutive image uploads were tested.
- The system handled repeated API calls without crashing.
- Session state maintained stability during extended usage.

6.2.3 System Reliability

- No UI freezing observed during API calls.
- Proper loading indicators were displayed during processing.
- Error messages were shown when API quota limits were exceeded.

7. RESULTS

AutoSage successfully:

- Identified vehicles from images.
- Generated structured automotive insights.
- Estimated resale value.
- Provided Indian market pricing context.
- Supported interactive advisory features.

Output ScreenShots

7.1 AutoSage UI:

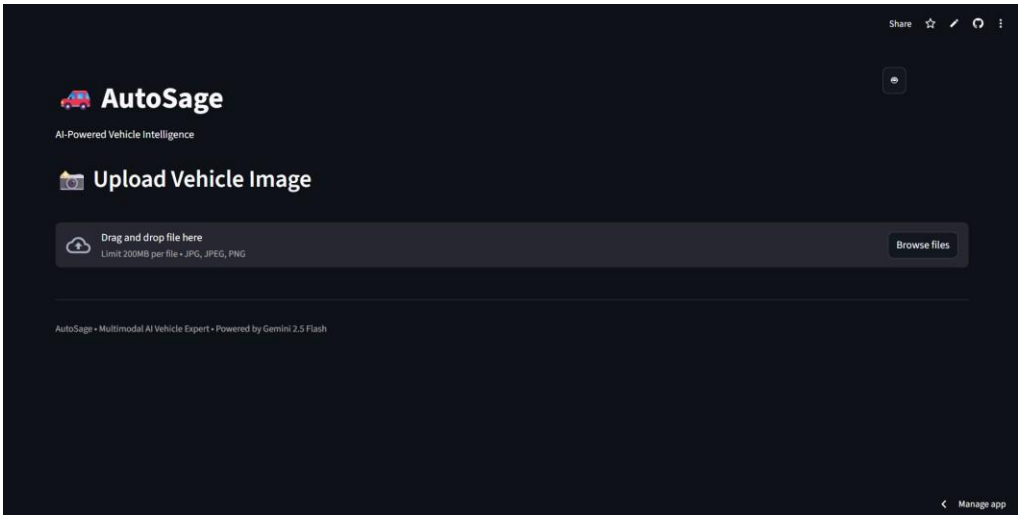


Figure 7.1.1: Image-Based Vehicle Analysis Interface allowing users to drag and drop vehicle images for intelligent insights.

Figure 7.1.2: Assistant Navigation Icon enabling access to the AutoSage Conversational Assistant module.

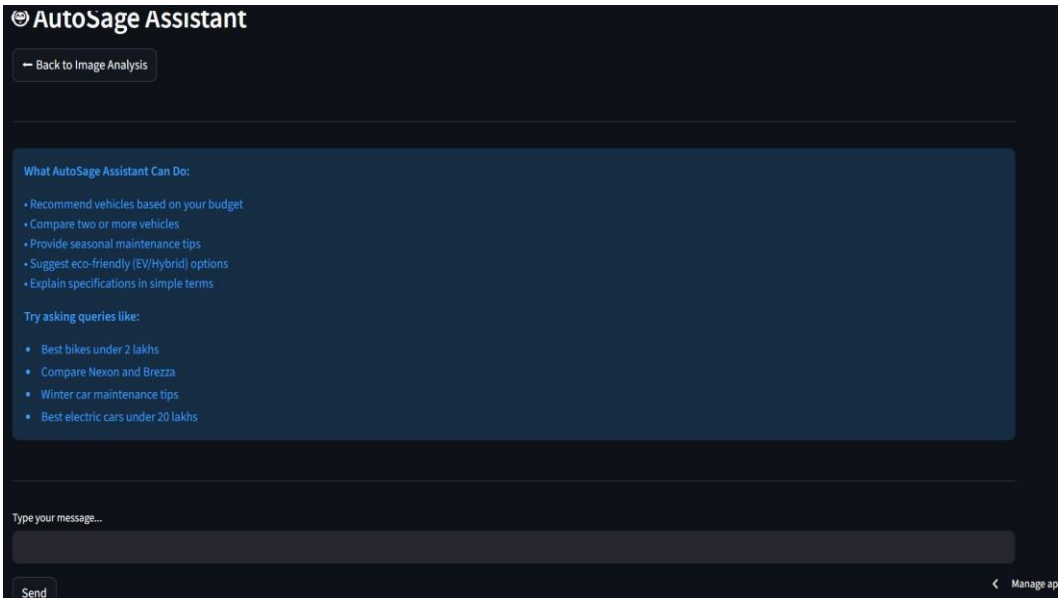


Figure 7.1.3: AutoSage Conversational Assistant Interface for interactive vehicle-related queries.

7.2 Primary Module – Image-Based Vehicle Analysis

Share

AutoSage

AI-Powered Vehicle Intelligence

Upload Vehicle Image

Drag and drop file here
Limit 200MB per file • JPG, JPEG, PNG

Browse files

image3.jpeg 20.3KB

Manage app

Analyze Vehicle

Vehicle Analysis

Brand: Land Rover

Model: Defender

Launch Year: 2020 (in India)

Manage app

Model: Defender

Launch Year: 2020 (in India)

Vehicle Type: SUV

Fuel Type: Petrol / Diesel / Hybrid

Key Features:

- Engine Capacity: 2.0L to 5.0L
- Transmission Type: 8-speed Automatic
- Top 3 Special Features:
  - Terrain Response 2 System
  - Electronic Air Suspension
  - Pivi Pro Infotainment System

Mileage: (Estimated) 9-12 km/l (depending on engine and fuel type)

Performance:

- Power Output: (Estimated) 300 PS to 525 PS
- Torque: (Estimated) 400 Nm to 650 Nm

Average Price in INR: ₹93 Lakhs - ₹2.35 Crores (Ex-showroom)

Maintenance Level: High

Safety Features:

- Multiple Airbags
- ABS with EBD
- Electronic Stability Control (ESC)
- Traction Control System (TCS)
- Roll Stability Control (RSC)
- Hill Descent Control (HDC)
- Emergency Brake Assist
- All-Terrain Progress Control (ATPC)
- 360-degree Camera

Other Details:

- Maintenance cost (approx): (Estimated) ₹50,000 - ₹1,00,000 per year for routine service
- Unique selling points: Iconic design, unparalleled off-road capability, luxurious interiors, high level of customisation.
- Target audience: Affluent individuals, adventure enthusiasts, luxury SUV buyers seeking ruggedness and sophistication.

Approximate Resale Value: (Estimated) ₹25 Lakhs - ₹45 Lakhs after 10 years

The Land Rover Defender stands as a highly capable and luxurious off-road SUV in the Indian market. It offers a unique blend of iconic heritage, advanced technology, and formidable performance for discerning buyers.

AutoSage • Multimodal AI Vehicle Expert • Powered by Gemini 2.5 Flash

Figure 7.2.1 Image-Based Vehicle Analysis producing structured AI-generated output for fourwheeler.

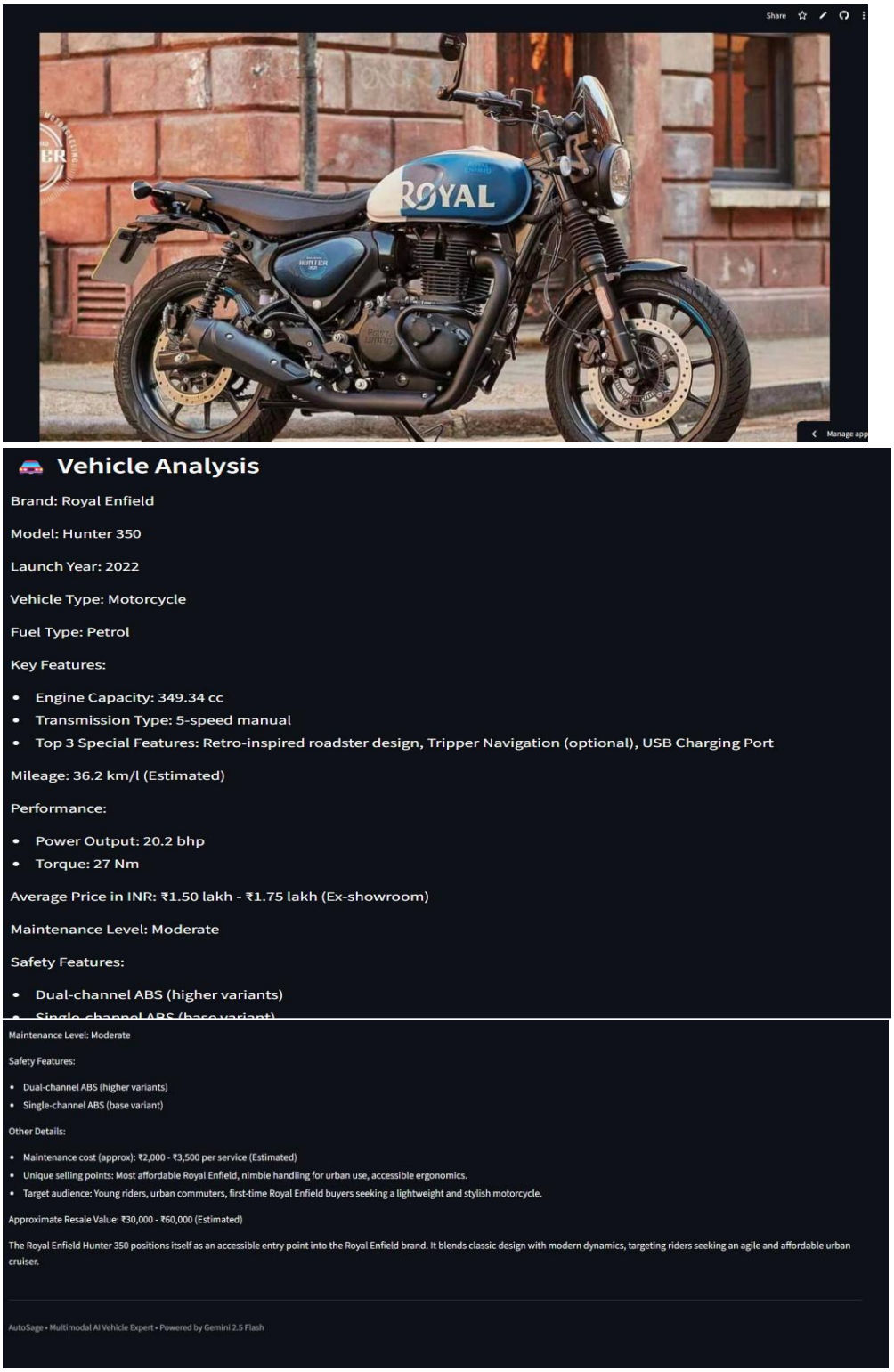


Figure 7.2.2 Image-Based Vehicle Analysis producing structured AI-generated output for twowheeler.

7.3 Enhancement Module – AutoSage Conversational Assistant

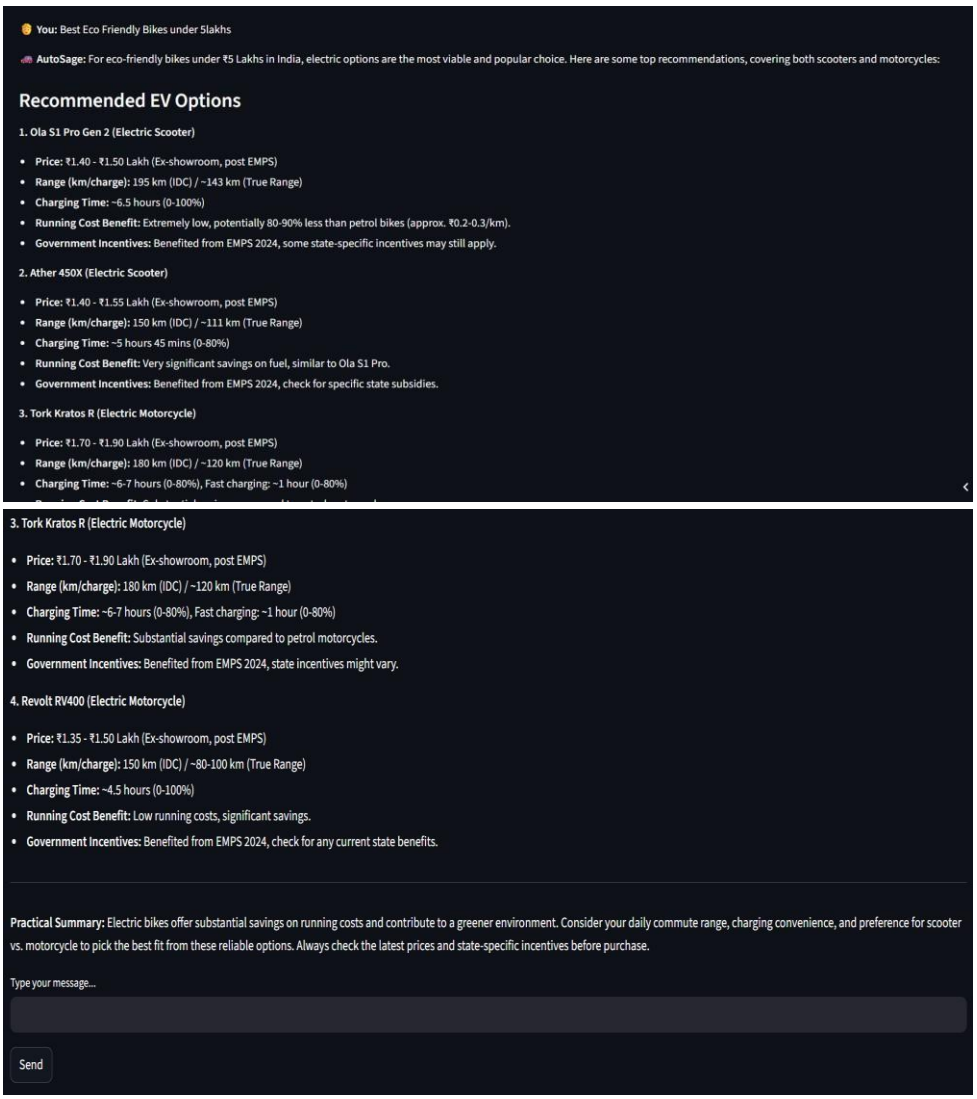


Figure 7.3.1: Conversational Assistant generating structured recommendations based on user vehicle queries

8. ADVANTAGES AND DISADVANTAGES

8.1 Advantages

1. Multimodal AI-Based System

AutoSage supports both image and text inputs using Gemini 2.5 Flash. This allows vehicle identification from images along with conversational advisory support, making the system more interactive and intelligent.

2. Structured Professional Output

The system generates well-formatted, section-wise responses including specifications, pricing, mileage, maintenance level, and resale value. This improves readability and decision-making clarity.

3. Indian Market Focused Insights

Outputs are tailored to the Indian automobile market, including INR pricing, mileage context, and EV considerations relevant to Indian users.

4. Scalable Architecture

The application uses API-based AI integration, making it easily deployable and extendable with future features.

5. No Custom Dataset Required

The system leverages a pre-trained generative AI model, eliminating the need for manual data collection or model training.

8.2 Disadvantages

1. API Quota Limitations

The system depends on Gemini API usage limits. Free-tier quotas may restrict the number of daily requests.

2. Internet Dependency

Since the model runs on cloud infrastructure, a stable internet connection is required for operation.

3. AI Estimation Variability

Some outputs such as pricing or resale value are estimated and may not always be exact.

4. Response Time

Average response time is 8–12 seconds due to cloud-based image processing and AI inference.

9. CONCLUSION

AutoSage successfully demonstrates the practical implementation of multimodal generative AI in the automotive domain. By integrating Google Gemini 2.5 Flash with a structured prompt engineering approach, the system effectively performs image-based vehicle identification and generates structured automotive insights relevant to the Indian market.

The primary module enables users to upload vehicle images and receive organized information such as specifications, pricing range, mileage, maintenance level, safety features, and estimated resale value. This structured approach enhances clarity and supports informed decisionmaking. Additionally, the integrated conversational assistant serves as a value-added enhancement by providing vehicle recommendations, comparisons, maintenance guidance, and eco-friendly vehicle suggestions. This dualmodule architecture ensures both visual intelligence and interactive advisory capability within a single platform.

Overall, the project validates the practical integration of Gemini Flash in decision-support systems and highlights the potential of multimodal AI in real-world applications.

10. FUTURE SCOPE

The current implementation of AutoSage can be further enhanced in several ways to improve accuracy, scalability, and user experience:

1. Real-Time Market Price API Integration

Integrating live automotive pricing APIs would allow the system to provide up-to-date and location-specific vehicle price ranges.

2. VIN Number Recognition

By incorporating VIN (Vehicle Identification Number) decoding, the system could provide exact vehicle variant details instead of estimations.

3. Vehicle Damage Detection

Computer vision techniques could be added to detect scratches, dents, or visible damage in used vehicle images, improving resale evaluation accuracy.

4. Insurance Estimation Module

An insurance estimation feature could calculate approximate premium costs based on vehicle type and market segment.

5. Mobile Application Deployment

Developing a mobile version of AutoSage would improve accessibility and real-time usage for buyers and sellers.

6. Multi-Language Support

Supporting regional languages would make the system more inclusive and accessible to a broader audience.

7. Voice-Enabled Assistant
Integrating speech-to-text and text-to-speech capabilities would allow users to interact with AutoSage through voice commands.

With these enhancements, AutoSage has the potential to evolve into a comprehensive AI-powered automotive intelligence platform.

11. APPENDIX

11.1 Project Repository

- **GitHub Link:** https://github.com/eedaratejaswini-07/autosageapp

11.2 Live Application Deployment:

- **Streamlit App:** https://autosage-appgp4curmwapppp2edzadnwdta.streamlit.app/

11.3 Video Demonstration

- **Project Demo Video:** https://drive.google.com/file/d/1rrwIvtJkw4qBJb4GAjfqua2YbGsxRw4e/view?usp=drive_link

11.4 Gemini API Documentation

- **Gemini API Documentation:** https://ai.google.dev/
- **Gemini Model Reference:** https://ai.google.dev/geminiapi/docs/models

11.5 Code Overview

11.5.1 Requirements File

```
requirements.txt
streamlit
google-generativeai
python-dotenv
Pillow
```

The following libraries are required to run the AutoSage application. These dependencies are listed in the requirements.txt file.

11.5.2 Environment Configuration

```
.env   requirements.txt   .gitignore
.env
GOOGLE_API_KEY=your_google_api_key_here
```

The application uses an environment configuration file (.env) to securely store sensitive credentials such as the Google API key. This prevents exposure of confidential information within the source code.

11.5.3 Core Code Snippets

1. API Initialization

This code initializes the Google Gemini API securely by retrieving the API key either from Streamlit secrets (cloud deployment) or from a local .env file. It ensures secure configuration without exposing sensitive credentials in the source code.

```
# 1 Initialization of Google API Key
# ==============================================
from dotenv import load_dotenv
import os
import streamlit as st
import google.generativeai as genai
from PIL import Image
# Load environment variables
try:
    # Try Streamlit Cloud Secrets
    api_key = st.secrets["GOOGLE_API_KEY"]
except Exception:
    # Fallback to local .env
    load_dotenv()
    api_key = os.getenv("GOOGLE_API_KEY")

# Configure Gemini API
genai.configure(api_key=api_key)
```

2. Model Loading Function

This function loads the pre-trained Gemini 2.5 Flash model. It establishes the connection between the application and the AI engine, enabling multimodal content generation for both image and text inputs.

```
# ==========================================
# 2️⃣ Interfacing With Pre-Trained Model
# ==========================================

def load_model():
    return genai.GenerativeModel("models/gemini-2.5-flash")
```

3. Prompt Engineering (Image-Based)

The prompt defines strict formatting rules and domain constraints to ensure the AI generates structured, automotive-specific output. This improves consistency, accuracy, and relevance of responses.

```
image_prompt = """
You are an automobile expert tasked with providing a detailed overview of any vehicles.

FORMATTING RULES:
- Follow the structure exactly as written below.
- Each section must be on a new line.
- Do NOT combine multiple fields in one sentence.
- Use bullet points properly (one point per line).
- If unsure, mention (Estimated).
- Keep the output crisp, practical, and structured.

The information should be presented in a structured format as follows:

Brand: Name of the vehicle brand.

Model: Specific model of the vehicle.

Launch Year: Since when the Vehicle is available in market.

Vehicle Type: (Scooter / Motorcycle / Sedan / SUV / Hatchback / Electric / Hybrid)

Fuel Type: (Petrol / Diesel / Electric / Hybrid)

Key Features:
- Engine Capacity:
- Transmission Type:
- Top 3 Special Features:

Mileage: Provide the average mileage in km/l (or km/charge for EVs).

Performance:
- Power Output (if known):
- Torque (if known):

Average Price in INR: Mention the price range of the vehicle model.

Maintenance Level: (Low / Moderate / High)

Safety Features:
```

```
Safety Features:
- ABS / Airbags / Stability Control / etc. (if applicable)

Other Details:
- Maintenance cost (approx):
- Unique selling points:
- Target audience:

Approximate Resale Value: Estimate the resale value of the vehicle after 10 years in Indian Rupees.

End with a short 2-line summary of the vehicle's overall positioning in the Indian market.
"""
```

4. Image Processing and AI Response Generation

This module handles the complete image analysis workflow. It reads the uploaded vehicle image, extracts its byte data and MIME type, and formats it into a structure compatible with the Gemini API. The processed image data is then sent to the pre-trained

Gemini model along with the structured automotive prompt. The model analyzes the input and generates a detailed, structured vehicle assessment as output.

```python
# Implement Function To Read Image
# ==========================================
def input_image_setup(uploaded_file):
    if uploaded_file is not None:
        bytes_data = uploaded_file.getvalue()
        return {
            "mime_type": uploaded_file.type,
            "data": bytes_data
        }
    else:
        raise FileNotFoundError("No file uploaded")
# ==========================================
#  Implement Function To Get Gemini Response
# ==========================================
def get_image_response(image_data):
    model = load_model()
    response = model.generate_content([image_prompt, image_data])
    return response.text
```

5. User Interface Integration

This code segment manages the user interaction workflow for image-based vehicle analysis. It enables image upload, displays the selected image, and triggers AI processing upon user request. The module connects the frontend interface with the backend AI engine, ensuring seamless execution of vehicle identification and structured analysis.

```python
# ==========================================
# PRIMARY FEATURE - IMAGE ANALYSIS
# ==========================================

if st.session_state.mode == "image":

    st.markdown("## 📷 Upload Vehicle Image")

    uploaded_file = st.file_uploader("", type=["jpg", "jpeg", "png"])

    if uploaded_file:
        image = Image.open(uploaded_file)
        st.image(image, width="stretch")

        if st.button("Analyze Vehicle", type="primary"):
            with st.spinner("Analyzing Vehicle..."):
                image_data = input_image_setup(uploaded_file)
                result = get_image_response(image_data)

            st.markdown("### 🚗 Vehicle Analysis")
            st.markdown(result)
```

6. Text-Based Assistant (Enhancement Module)

This module processes user text queries related to vehicles, constructs structured prompts, and generates advisory responses. It enhances the system by providing recommendations, comparisons, and maintenance guidance.

```
# Prompt for chat assistant
def build_prompt(user_query):
    return f"""
You are AutoSage, an expert AI automotive advisor focused on the Indian automobile market.

Your role:
Provide clear, structured, and practical vehicle insights to help users make informed decisions.

FORMATTING RULES:
- Keep responses crisp and easy to scan.
- Use bullet points where helpful.
- Do NOT write long paragraphs.
- Provide realistic price and mileage ranges.
- If uncertain, provide approximate values.
- Focus on decision-making insights.

User Query:
{user_query}

---------------------------------------
RESPONSE STRUCTURE (Use as applicable)
---------------------------------------

If the query is about Buying / Recommendation:

Recommended Models (2-4 options):
- Model Name - Price Range (INR)

For each model include:
- Mileage:
- Key Features (Top 3):
- Best For:

Short Verdict: Which option is better and why.

---------------------------------------

If the query is about Comparison:
```

```
Short Verdict: Which option is better and why.

---------------------------------------

If the query is about Comparison:

Comparison Overview:
- Price Difference:
- Mileage Difference:
- Feature Highlights:
- Pros & Cons (each model)

Recommendation: Which one to choose and for what type of buyer.

---------------------------------------

If the query is about Maintenance:

Maintenance Advice:
- Key Checks:
- Seasonal Tips (if relevant):
- Estimated Service Cost Level:

---------------------------------------

If the query is about Eco-Friendly Vehicles:

Recommended EV/Hybrid Options:
- Model - Price - Range (km/charge)

Include:
- Charging Time:
- Running Cost Benefit:
- Government Incentives (India context if applicable)

End every response with a short 2-3 line practical summary.
"""
```

```
def get_text_response(prompt):
    model = load_model()
    response = model.generate_content(prompt)
    return response.text
```